

## Throughput e Latência com Apache Bench

---

### Objetivo

Medir o desempenho de um servidor web sob diferentes níveis de carga, coletando métricas de throughput e latência. O aluno irá simular requisições HTTP com concorrência variada, observar o comportamento do servidor e visualizar os resultados no Datawrapper.

### Contexto: Throughput e Latência em sistemas web

Em sistemas web, as duas métricas mais importantes são:

- Throughput — quantas requisições o servidor consegue atender por segundo. Medido em req/s. Quanto maior, melhor.
- Latência — quanto tempo o servidor leva para responder uma requisição. Medido em milissegundos. Quanto menor, melhor.

A concorrência é o número de requisições simultâneas que chegam ao servidor. Quanto maior a concorrência, mais o servidor precisa dividir seus recursos — e o impacto no throughput e na latência revela os limites do sistema.

Neste laboratório também medimos:

- Percentil 90 (p90) — 90% das requisições foram respondidas em até esse tempo
- Percentil 99 (p99) — 99% das requisições foram respondidas em até esse tempo

Os percentis são mais importantes que a média em sistemas reais: um servidor pode ter latência média de 10ms mas p99 de 500ms — o que significa que 1% dos usuários esperam muito mais que os outros.

### Pré-requisitos

- Docker e Docker Compose instalados
- Conhecimento básico de terminal Linux

### Como iniciar o laboratório

Abra um terminal nesta pasta e execute:

```
docker compose up -d
docker compose exec avd-lab-02 bash --login
```

## Exercícios

### Exercício 1 — Iniciar o servidor web e verificar o funcionamento

O container já vem com o Apache instalado. Inicie o servidor:

```
service apache2 start
```

Verifique se está respondendo:

```
curl http://localhost/
```

A resposta deve mostrar o HTML da página de teste. Isso confirma que o servidor está pronto para receber requisições.

### Exercício 2 — Entendendo o Apache Bench

O Apache Bench (ab) é uma ferramenta de benchmark HTTP que simula múltiplos usuários acessando um servidor ao mesmo tempo. Execute um teste simples:

```
ab -n 100 -c 1 http://localhost/
```

Entendendo os parâmetros:

- -n 100 — total de 100 requisições a serem feitas
- -c 1 — apenas 1 requisição por vez (sem concorrência)

Na saída, observe:

- Requests per second — throughput: quantas requisições por segundo
- Time per request — latência média em milissegundos
- Percentage of requests served within a certain time — distribuição de latência (p50, p90, p99)

### Exercício 3 — Executar o benchmark automatizado

O script benchmark\_web.sh roda 3 cenários de carga e salva os resultados:

```
./benchmark_web.sh
```

Os 3 cenários testados são:

| Cenário | Requisições | Concorrência |
|---------|-------------|--------------|
|---------|-------------|--------------|

|             |     |                |
|-------------|-----|----------------|
| Carga Leve  | 100 | 1 simultânea   |
| Carga Média | 100 | 10 simultâneas |
| Carga Alta  | 100 | 50 simultâneas |

O que esperar? Com mais concorrência, o throughput pode aumentar até certo ponto — depois começa a cair porque o servidor fica sobrecarregado. A latência tende a aumentar com mais concorrência.

## Exercício 4 — Verificar o CSV gerado

```
cat resultados/benchmark_web.csv
```

O formato é:

```
cenario,requisicoes,concorrencia,req_por_seg,latencia_media_ms,latencia_p90_ms,latencia_p99_ms
Carga Leve,100,1,...
Carga Media,100,10,...
Carga Alta,100,50,...
```

Copie toda a saída — você vai usar no Exercício 6.

## Exercício 5 — Observar o servidor durante a carga

Abra um Terminal 2 nesta pasta e entre no container:

```
docker compose exec avd-lab-02 bash --login
```

No Terminal 2, monitore o sistema:

```
htop
```

No Terminal 1, rode o benchmark novamente:

```
./benchmark_web.sh
```

Observe no Terminal 2: o uso de CPU durante cada cenário, como a carga muda conforme aumenta a concorrência e quais processos estão consumindo mais recursos.

## Exercício 6 — Visualizar no Datawrapper

Passo 1 — Acesse [datawrapper.de](https://datawrapper.de) e clique em Start creating.

Passo 2 — Clique em Copy & paste data table e cole o conteúdo completo do CSV.

Passo 3 — Clique em Proceed até chegar em Visualize.

Passo 4 — Escolha Column Chart.

Passo 5 — Na aba Refine → Select column, explore cada métrica:

- req\_por\_seg — throughput por cenário
- latencia\_media\_ms — latência média
- latencia\_p90\_ms — latência no percentil 90
- latencia\_p99\_ms — latência no percentil 99

O que observar:

- O throughput aumentou ou caiu com mais concorrência?
- A diferença entre latência média e p99 é grande? O que isso indica?
- Qual cenário representa o melhor equilíbrio entre throughput e latência?

## Exercício 7 — Análise dos resultados

Com base nos gráficos, responda:

1. O throughput aumentou linearmente com a concorrência?
2. Qual foi o impacto no p99 ao passar de carga leve para alta?
3. Existe um ponto onde aumentar a concorrência prejudica o desempenho?
4. O que a diferença entre latência média e p99 revela sobre o comportamento do servidor?
5. Se esse fosse um servidor em produção, qual nível de concorrência você configuraria como limite?

Conceito: Em teoria das filas, esse comportamento é descrito pela Lei de Little: o número médio de requisições no sistema é igual ao throughput multiplicado pela latência média. Quando a concorrência sobe além da capacidade do servidor, a latência explode — é o ponto de saturação do sistema.

## Como encerrar o laboratório

```
exit  
docker compose down
```

*Os resultados ficam salvos na pasta resultados/ mesmo após encerrar o container.*